CS 422 Assignment 3: TCP Congestion Control Implementation

Option 1: Linux Kernel Module Implementation Name: Kush Khanna **Repository:** https://github.com/krash3125/cs422-assignment-2 (branch `assignment-3`)

> All measurements, tables and plots below come from one automated run of `./run_a3.sh`, stored under `results_a3/20261024-151559/`. Public Internet paths change between runs, so a re-run reproduces the method and the qualitative findings, not the exact numbers.

---

1. QAIMD Implementation & Congestion-Window Update Logic

`tcp_qaimd.c` is a loadable Linux congestion-control module implementing the **Queue-Aware AIMD with a BDP Anchor (QAIMD)** algorithm designed in Assignment 2, Question 3(b). Rather than relying purely on buffer overflow (CUBIC, Reno) or ignoring packet loss entirely (BBR v1), QAIMD anchors `snd_cwnd` to the estimated bandwidth-delay product (BDP = btl_bw × min_rtt) and regulates standing queues using a per-round minimum-RTT signal.

The module registers with the kernel as congestion control `"qaimd"` via `tcp_qaimd_ops` and is selected on the data socket by the Assignment-2 Python client with `setsockopt(IPPROTO_TCP, TCP_CONGESTION, b"qaimd")` *before* `connect()`; the client reads the value back, so every logged sample records `cc_used = qaimd` (verified: `cc_requested == cc_used` in all 94 successful tests). Because QAIMD needs full control of the window in every state (like BBR), it uses the `.cong_control` hook rather than `.cong_avoid`, so the kernel's PRR reduction is bypassed and the module owns every `cwnd` update.

How `cwnd` updates from RTT, loss proxy, and throughput:

1. **Per-ACK signal collection** — `qaimd_main`:
   - **RTT.** On each ACK carrying an RTT sample, the module tracks `rtt_round_min_us` (minimum RTT in the current round) and `min_rtt_us` (windowed minimum over `QAIMD_PROBE_RTT_MS = 10 s`). The standing queue is $q = \text{rtt_round_min_us} - \text{min_rtt_us}$; using the per-round minimum filters transient Wi-Fi delay spikes that prematurely trip CUBIC's HyStart.
   - **Throughput / delivery rate.** On non-app-limited ACKs it computes the instantaneous delivery rate (B/s) and keeps a **time-windowed maximum** `btl_bw` over ~10 round trips, then $\text{bdp} = \max\left(\frac{\text{btl_bw} \times \text{min_rtt_us}}{\text{MSS}}, 4\right)$ (`qaimd_bdp_segs`).
   - **Loss proxy.** Across each round it tracks Δretrans and Δdelivered from the kernel's segment counters. A loss is **congestive** only if Δretrans > 0 **and** either $q > q_{\text{thr}}$ or the per-round loss ratio exceeds 2.0%. Isolated non-congestive losses (no standing queue) are ignored.
2. **Per-round window update** — `qaimd_update_Round`:
   - **STARTUP:** `cwnd` doubles each round until (a) congestive loss, (b) $q > 2q_{\text{thr}}$ for 2 rounds, or (c) `btl_bw` grows < 25% for 3 rounds. On exit, `cwnd` drops directly to max(bdp, 4) to drain the startup overshoot.
   - **AVOID (congestion avoidance):**
     - *Congestive loss:* multiplicative decrease cwnd ← max(0.7 cwnd, 4).
     - *Queue above threshold:* proportional delay back-off cwnd ← $\max(\text{cwnd}\,(1 - 0.5\frac{q - q_{\text{thr}}}{\text{srtt}}),\ 4)$, capped so the cut is never harsher than the loss cut.
     - *Sub-BDP catch-up (cwnd < bdp):* cwnd ← cwnd + max(1, (bdp − cwnd)/4), so long-RTT paths recover in a few RTTs instead of hundreds of Reno steps.
     - *Probing (cwnd ≥ bdp, $q \le q_{thr}$):* additive increase +1 MSS per round.
   - **ProbeRTT:** when `min_rtt` has not been refreshed for 10 s, the module drains to max(0.5 bdp, 4) for max(200 ms, 1 srtt) to re-measure propagation delay, then restores the window.
   - **RTO** (`qaimd_set_state`): `cwnd = 4`, re-enter STARTUP with `ssthresh` capped at the last BDP so it does not overshoot again.

Pacing is set every ACK to 1.1 × cwnd · MSS/srtt (`qaimd_set_pacing_rate`).

---

2. Initial State, Parameters, and Deviations from Pseudocode

Parameter / state	Value in `tcp_qaimd.c`	Purpose
Initial `state`	`QAIMD_STARTUP` (0)	Exponential bandwidth search at flow start
Initial `cwnd`	`10 MSS` (kernel default)	Baseline starting window
`QAIMD_BETA`	`700/1000` (0.7)	Multiplicative decrease on congestive loss
`QAIMD_LOSS_TOL`	`20/1000` (2.0%)	Loss-ratio threshold when $q \le q_{\text{thr}}$
`QAIMD_MIN_CWND`	`4 MSS`	Congestion-window floor
`QAIMD_PROBE_RTT_MS`	`10 000 ms`	`min_rtt` window / ProbeRTT trigger
`q_thr`	`clamp(0.15·min_rtt, 3 ms, 15 ms)`	Target standing-queue delay (`qaimd_q_thr_us`)
`QAIMD_PACE_GAIN`	`110/100` (1.1×)	Pacing-rate multiplier

Initial state is set in `qaimd_init`: `min_rtt_us = ~0`, `btl_bw = 0`, round markers seeded from `tp->delivered` and `tp->total_retrans`, and kernel-internal pacing enabled if `fq` is not in use.

Changes from the Assignment-2 pseudocode:

1. **Fixed-point integer arithmetic.** The kernel forbids floating point in socket context, so every constant (0.7, 0.15, 1.1, 0.5, 0.02) is implemented in scaled integer / microsecond / bytes-per-second units.
2. **Packet-count loss proxy.** `tp->total_retrans` and `tp->delivered` are kept in segments; since MSS is constant, Δretrans/Δdelivered equals the byte-ratio the pseudocode specified, while staying in 32-bit arithmetic.
3. **Time-windowed-max `btl_bw`.** The pseudocode specified a maximum over the last 10 round trips. Rather than store a ring buffer inside the 104-byte `ICSK_CA_PRIV_SIZE` limit, `btl_bw` is a single peak with a `btl_bw_stamp`: any new sample $\ge$ the peak replaces it, and a peak older than the window ($\approx 10 \cdot$ srtt, clamped to 0.2–10 s) is replaced by the current sample. *(This replaced an earlier per-ACK 0.995× decay that collapsed the estimate at high ACK rates.)*
4. **min_rtt refresh via ProbeRTT only.** `min_rtt` is updated only on a genuine new minimum; the 10 s staleness drives the ProbeRTT drain instead of silently overwriting `min_rtt` with a possibly-inflated current sample. This keeps `min_rtt` a true propagation-delay estimate and makes ProbeRTT actually fire.
5. **Capped proportional delay back-off.** The per-round delay reduction is capped at 30% so a Wi-Fi spike can never cut `cwnd` harder than the congestive-loss cut.
6. **`cong_control` hook signature.** The Ubuntu 24.04 / 6.8 kernel uses the two-argument `cong_control(sk, rs)` form (not the later `(sk, ack, flag, rs)` variant); QAIMD uses this form and never needs `ack`/`flag`.

---

3. Kernel Environment, Build, and Reproduction

- **Host:** macOS with a Multipass Ubuntu 24.04 VM (the same approach as Assignment 2), which provides a real kernel that can load a custom module — unlike Docker Desktop, whose shared VM kernel cannot `insmod` one.
- **Exact kernel:** `6.8.0-147-generic` (aarch64), **Ubuntu 24.04.5 LTS**. Default CC `cubic`, default qdisc `fq_codel`, `tcp_wmem = 4096 16384 4194304`. Recorded automatically in `results_a3/20261024-151559/metadata.json`
- **Kernel requirements:** `CONFIG_TCP_CONG_ADVANCED=y` and `tcp_bbr` available for the comparison runs. After loading, the preflight confirms all four algorithms are settable (`cc_check: {qaimd: ok, cubic: ok, reno: ok, bbr: ok}`), and `tcp_available_congestion_control` lists `reno cubic bbr qaimd`.
- **Create the Linux VM (macOS):**

```
brew install --cask multipass
multipass launch 24.04 --name cs422 --cpus 2 --memory 4G --disk 15G
multipass shell cs422
sudo apt-get update
```

```
sudo apt-get install -y build-essential linux-headers-$(uname -r) \
     python3-numpy python3-pandas python3-matplotlib git
```

- **Build & load** (`Makefile`; `KDIR` defaults to `/lib/modules/$(uname -r)/build`):

```
make            # builds tcp_qaimd.ko against the installed kernel headers
make load       # modprobe tcp_bbr; insmod tcp_qaimd.ko; show available CC
sysctl net.ipv4.tcp_available_congestion_control   # must list: ... bbr qaimd
```

  (An unsigned out-of-tree module taints the kernel — expected and harmless.)

- **Full experiment** (build + load + collect + analyze):

```
git clone https://github.com/krash3125/cs422-assignment-2.git
cd cs422-assignment-2 && git checkout assignment-3
sudo ./run_a3.sh          # writes results_a3/<RUN_ID>/{samples.csv,tests.jsonl,tables,plots}
make unload               # rmmod when finished
```

During grading we demonstrate that the module compiles and loads on this host; the grading environment does not load it.

---

4. Experimental Methodology

- **Destinations:** the geographically diverse public `iperf3` servers in `servers_a3.csv`. Of 10 requested, **8 were selected** (shortfall 2; recorded in `destinations.json` / `tables/q1_destinations.md`). The two that failed selection were `89.187.165.1` (Zürich, `ACCESS_DENIED` — busy) and `iperf-ams-nl.eranium.net` (Amsterdam, connection reset); see `tables/q1_failures.md`. Selected set:

Destination	Site	min RTT (ms)
speedtest.wdc2.us.leaseweb.net	Washington, US	24.7
66.35.22.79	Ashburn, US	25.2
speedtest.dal13.us.leaseweb.net	Dallas, US	34.0
84.17.41.11	Seattle, US	47.9
iperf3.moji.fr	Paris, FR	100.2
iperf.worldstream.nl	Naaldwijk, NL	101.9
speedtest.syd12.au.leaseweb.net	Sydney, AU	180.2
89.187.162.1	Singapore, SG	195.4

- **Duration & sampling:** every test runs for **10.0 s** with TCP statistics (cwnd, goodput, srtt / min_rtt / rttvar, retransmits, RTO events, pacing / delivery rate) sampled every **200 ms** — identical for all four algorithms.
- **Algorithms:** `qaimd` (our module), `cubic` (Linux default), `reno`, and `bbr` (Linux v1).
- **Loss signal logged:** per-interval retransmitted segments (`retrans_pkts_delta`), retransmitted bytes, RTO events (`rto_events_delta`), and `ca_state` (Recovery/Loss), all from `TCP_INFO`.
- **Repetitions & test order: 3 repetitions** of every destination × algorithm; the algorithm order is rotated each repetition so no algorithm always runs first on a path (metadata `test_order`: *for rep → for destination → algorithms in rotated order*):
  - *Rep 1:* `qaimd → cubic → reno → bbr`
  - *Rep 2:* `cubic → reno → bbr → qaimd`
  - *Rep 3:* `reno → bbr → qaimd → cubic`

  The four algorithms on a destination run back-to-back (within ~45 s) so they see similar path conditions.
- **Completion: 94 of 96 tests succeeded.** Two did not complete after all retries — one `qaimd` repetition to Sydney (repeated `CONNECT_TIMEOUT` / server not responding) and one `bbr` repetition to Ashburn — so

qaimd and bbr have 23 runs and cubic/reno have 24. End-to-end integrity held: median |bytes_acked − server_bytes|/bytes_acked = 0.04%.

- **Representative destination:** 84.17.41.11 (Seattle; min RTT about 48 ms), used for all per-destination time-series and scatter plots.

---

5. Results and Comparative Analysis

5.1 Aggregate throughput and RTT statistics

Pooled over all destinations and repetitions (tables/a3_algo_summary.md; percentiles use the pandas/NumPy linear-interpolation method, quantile(0.95)):

CC	runs	goodput mean (Mbps)	goodput median (Mbps)	goodput p95 (Mbps)	srtt median (ms)	RTT inflation (mean)	retrans % (mean)
qaimd	23	103.54	96.80	186.84	**67.75**	**1.09**	**0.24**
cubic	24	139.60	134.48	254.56	114.31	1.61	0.31
reno	24	144.19	146.68	253.75	104.15	1.54	0.63
bbr	23	157.61	151.25	282.99	114.54	1.68	0.43

The headline trade-off is clear: **QAIMD runs at the lowest RTT inflation (1.09× vs 1.5–1.7×) and the lowest retransmit rate (0.24%)**, at the cost of ~26–34% lower mean goodput than the loss-based algorithms and BBR. Per-destination detail is in tables/q3_dest_summary.md; the per-run table is tables/q3_per_run.md; per-destination throughput min/median/mean/p95 is in tables/q1_goodput_summary.md. The per-destination "winners" (tables/q3_winners.md) show **QAIMD gives the lowest srtt on 7 of 8 destinations**, while BBR or CUBIC win raw goodput on all of them.

5.2 Four-algorithm plots (representative destination: Seattle)

Time series of cwnd, RTT, loss proxy, and goodput, one column per algorithm, shared y-axis per row (A2 Q2(b)(i), repeated for all four algorithms):

The same four metrics overlaid for direct comparison:

Scatter plots of cwnd, srtt, and retransmissions against goodput (A2 Q2(b)(ii)), one column per algorithm, consistent axis scales:

Run-to-run variability and the throughput-vs-queueing/loss trade-off across all destinations:

5.3 Operating regimes and explanation of differences

(1) Short paths with deep bottleneck buffers (Washington, min RTT 24.7 ms; Ashburn 25.2 ms; Dallas 34.0 ms). Here the BDP is small and the loss-based algorithms overfill the buffer. At Washington, BBR/Reno/CUBIC reach 224/202/174 Mbps but inflate RTT by 2.46×/2.04×/2.45× (median srtt 77/64/79 ms). QAIMD holds the standing queue near q_thr: **srtt 36.5 ms, RTT inflation 1.13×, retransmit 0.01%, cwnd/BDP 0.82, in-flight 0.51 MB** (vs CUBIC's 2.20 MB). It trades throughput (107 Mbps) for a near-empty queue — exactly the design goal. Dallas and Ashburn show the same pattern (QAIMD srtt 40.6 / 52.5 ms vs 60–87 ms; inflation 1.12 / 1.18 vs 1.6–2.6).

(2) Representative middle ground (Seattle, min RTT 48 ms). CUBIC/Reno/BBR reach 200/194/197 Mbps at RTT inflation 1.79×/1.73×/2.15× (srtt 89/86/105 ms); QAIMD reaches 112 Mbps at **srtt 53.5 ms, inflation 1.08×, retransmit 0.03%**. The time-series grid shows CUBIC/Reno's sawtooth cwnd and rising srtt versus QAIMD's flatter cwnd held near the BDP and near-flat srtt; the scatter grid shows CUBIC/Reno goodput spread across inflated-RTT samples while QAIMD's cluster at low srtt.

(3) Long, thin paths with shallow queues (Sydney 180 ms, Singapore 195 ms). Here *every* algorithm shows RTT inflation about 1.0 — these paths are capacity-limited with little standing queue, so there is no

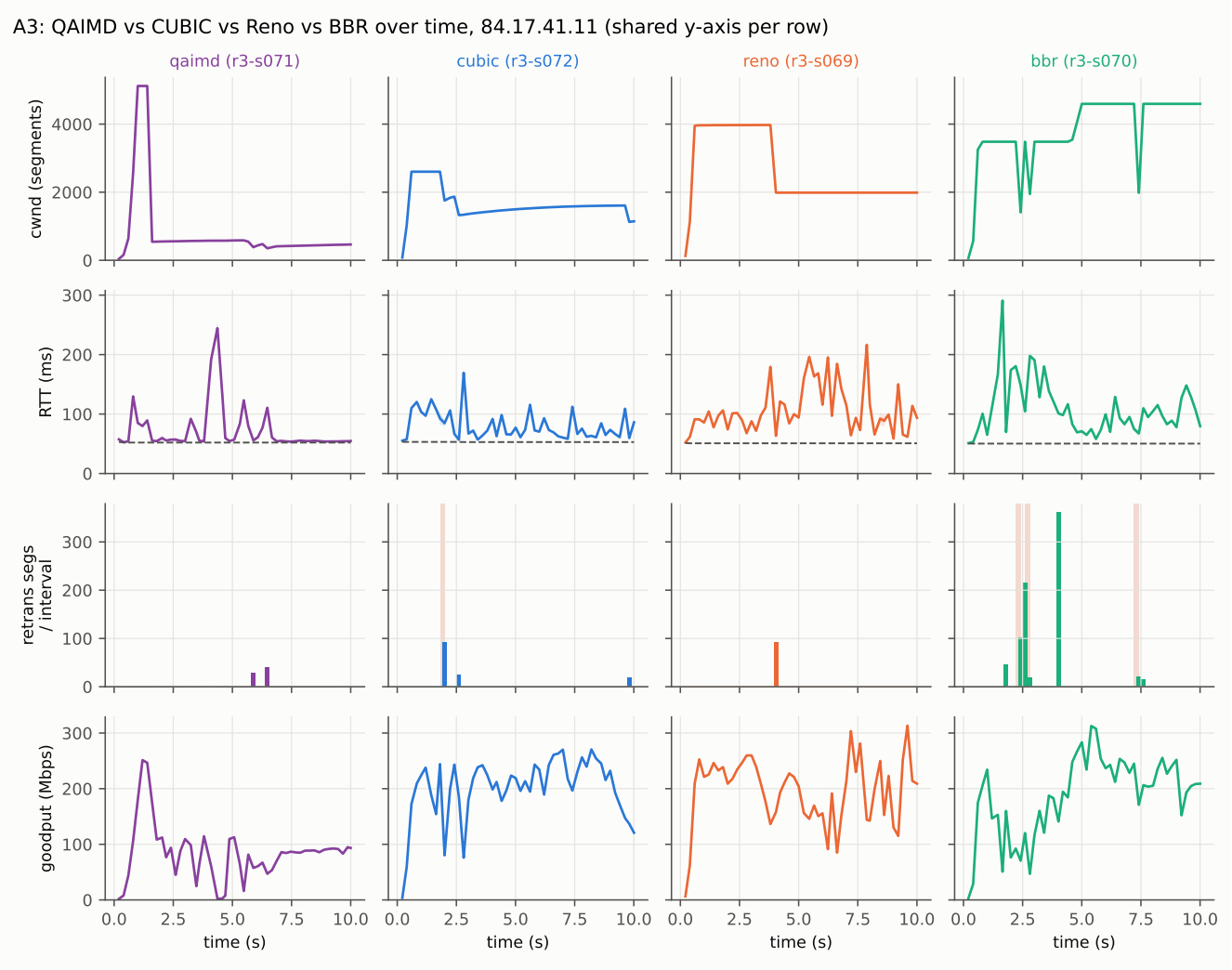


Figure 1: Time series — QAIMD vs CUBIC vs Reno vs BBR, Seattle, median run of each algorithm.

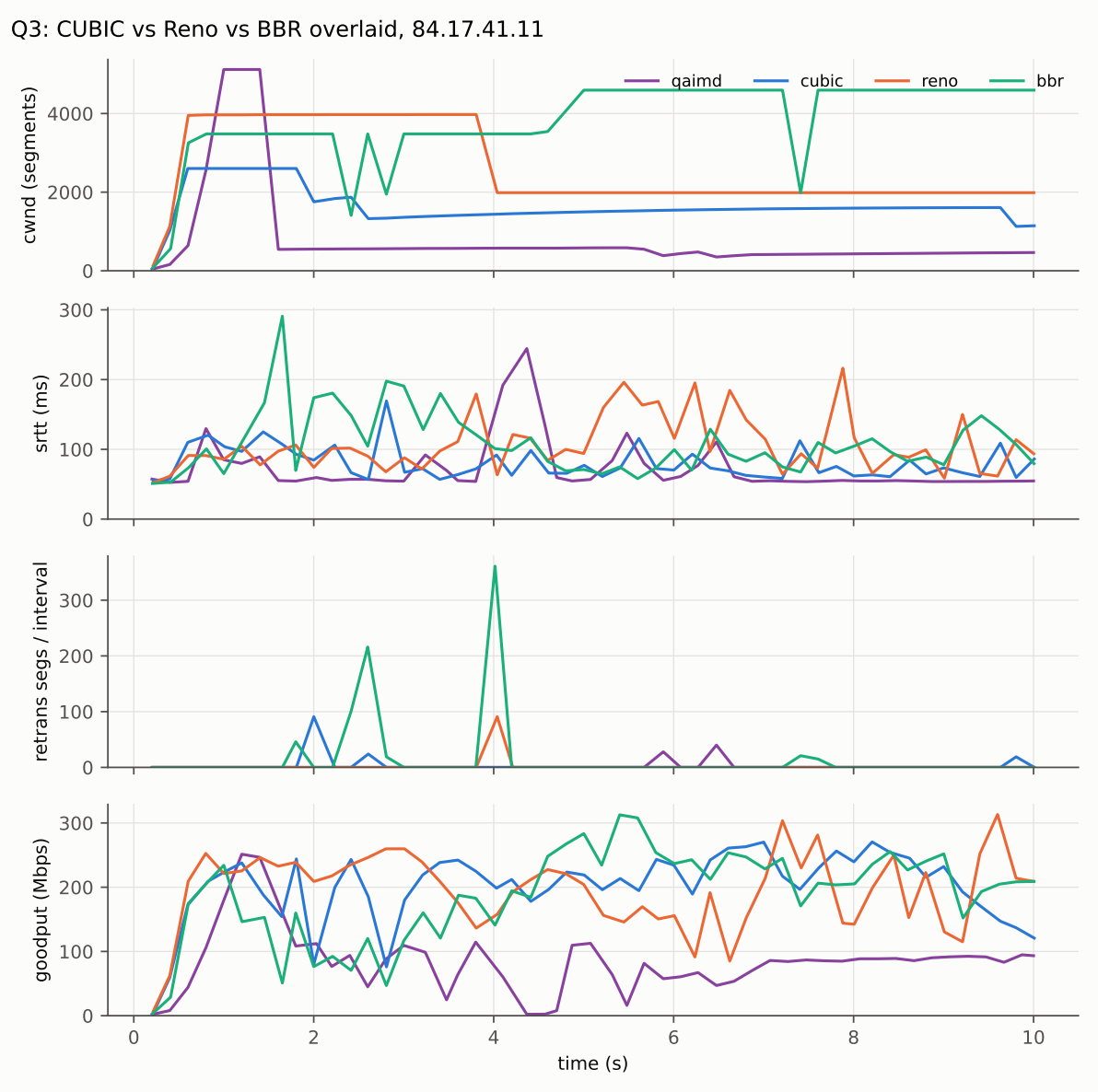


Figure 2: Overlay — the four algorithms on shared axes, Seattle.

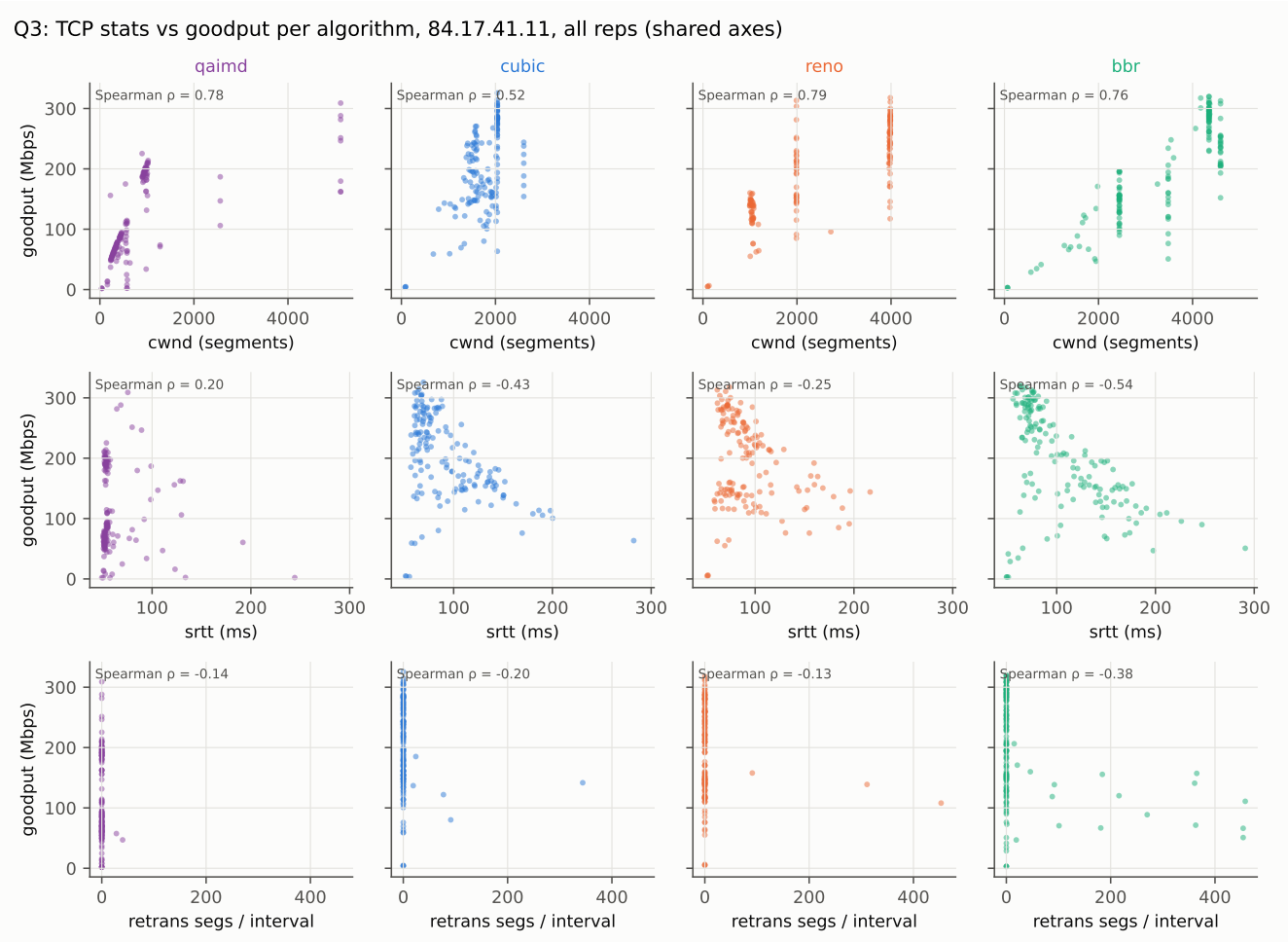


Figure 3: Scatter — cwnd / srtt / retransmits vs goodput, per algorithm, Seattle, all reps.

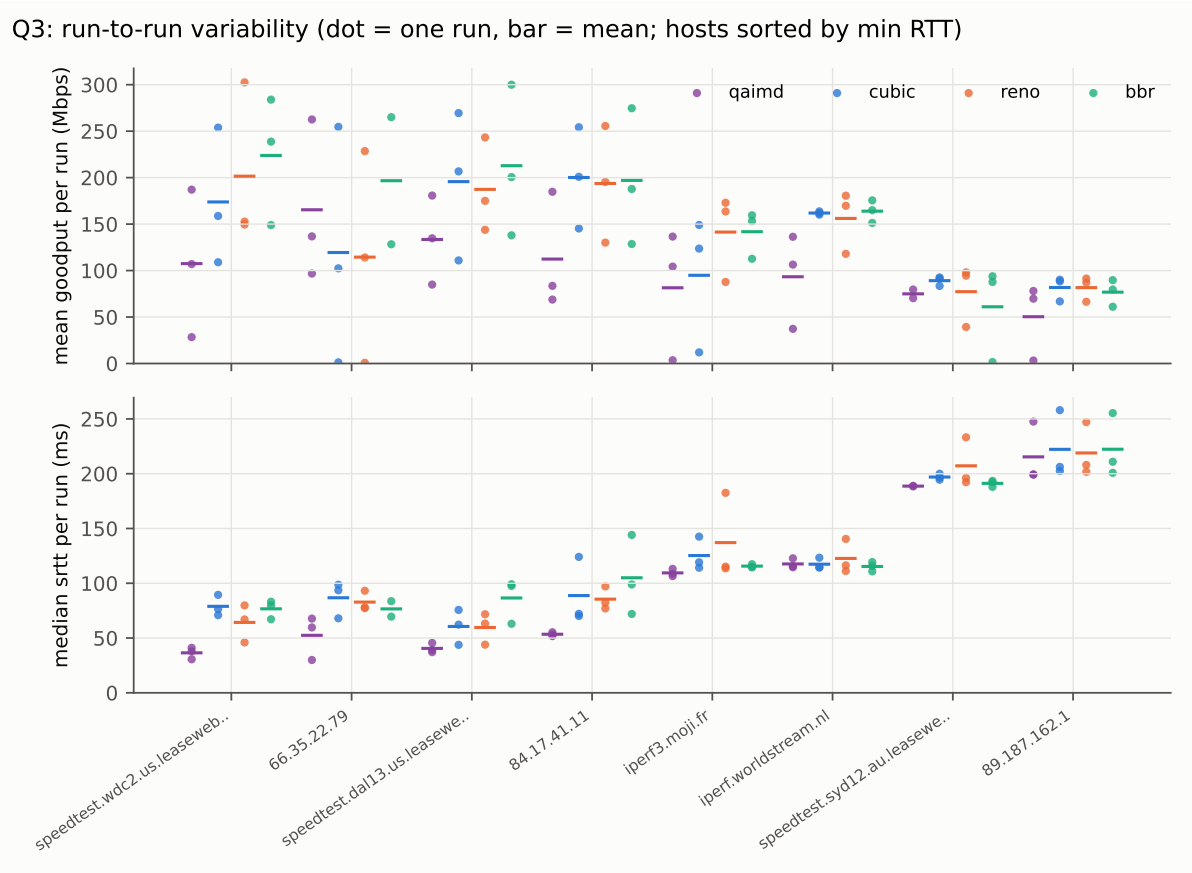


Figure 4: Variability — mean goodput and median srtt per run, hosts sorted by min RTT.

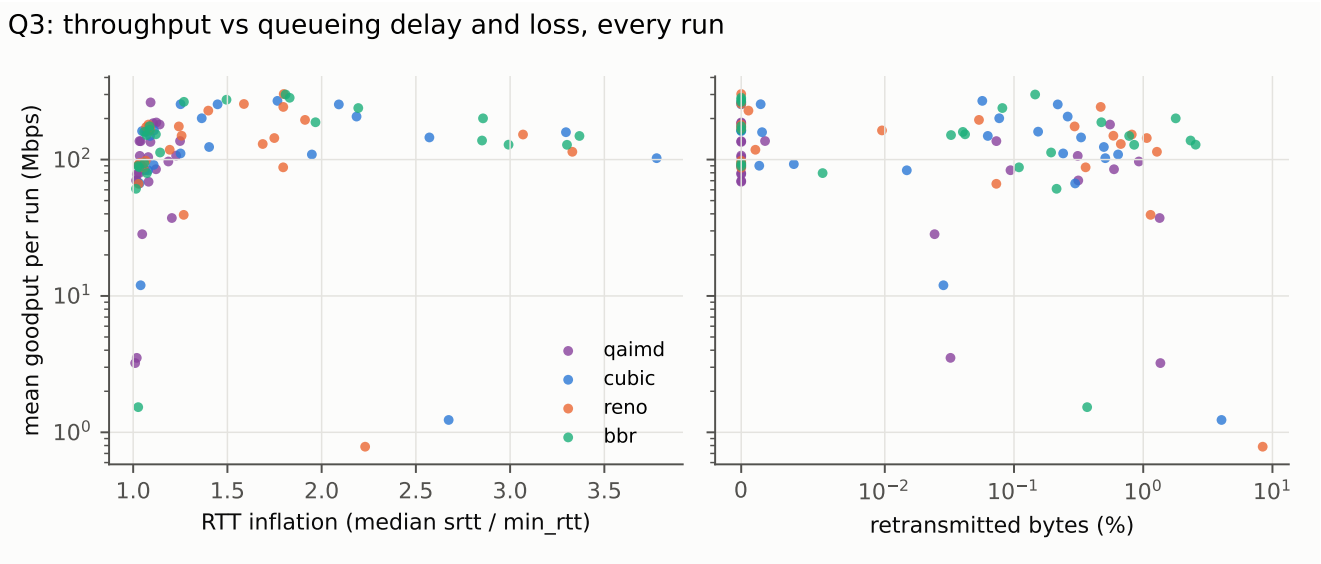


Figure 5: Trade-off — goodput vs RTT inflation and vs retransmit %, every run.

bufferbloat to avoid. QAIMD's latency advantage therefore disappears and its conservatism costs throughput: at Singapore it averages **50.3 Mbps vs ~82 Mbps** for the others (with high variance, std 41 Mbps), and at Sydney 74.9 vs CUBIC's 89.1. Over a 10 s test these paths are startup-dominated, and QAIMD's BDP-anchored exit underfills when `btl_bw` is still ramping. This is the clearest limitation: on a high-BDP path with no queue to manage, a delay-targeting controller gives up rate for no latency benefit.

Relation to cwnd, RTT, queueing, loss and the BDP. QAIMD keeps in-flight data near one BDP, so `cwnd/BDP about 1` and srtt about min_rtt; CUBIC/Reno push `cwnd` past the BDP until loss, so the excess sits in the queue and srtt rises (`cwnd/BDP` 1.3–3.7 on short paths). BBR targets the BDP too but probes more aggressively, giving the highest goodput and the highest retransmit rate (2.2–3.7% of bytes on short paths vs QAIMD's <1%).

5.4 Variability and limitations

- **Variability across runs.** Goodput standard deviation is large on the congested short paths for every algorithm (e.g. Washington std 69–87 Mbps) because of cross-traffic and when each flow happens to take a loss; it is much smaller on the long thin paths (Singapore std 13–14 Mbps for the loss-based algorithms). QAIMD is the most variable at Singapore (std 41 Mbps), reflecting the startup-sensitivity noted above. See `q3_variability.pdf`.
- **Public-path limits.** We cannot observe the servers' buffer depths or cross-traffic, so absolute numbers are path- and time-dependent; the comparison is valid only because the four algorithms run back-to-back per destination.
- **Delay-sensitivity / fairness.** QAIMD yields to standing queues, so a QAIMD flow sharing a deep-buffer bottleneck with CUBIC would be starved; its competitive mode (fall back to AIMD after 20 rounds of queue with shrinking cwnd and no loss) limits but does not eliminate this.
- **Short test duration.** 10 s per test (per the assignment) means long-RTT paths spend most of the test in startup, so steady-state behaviour is under-sampled there and the long-path conclusions are startup-dominated.

---

6. Use of AI Tools

I used Claude Code (Anthropic's CLI) as a pair programmer for this assignment. It helped translate the Assignment-2 QAIMD pseudocode into a kernel module, cross-check the `tcp_congestion_ops` interface and the `cong_control`/`ssthresh`/`set_state` contract against the 6.8 kernel source (including correcting the `cong_control` signature to the two-argument form this kernel uses), convert the floating-point design to fixed-point, and set up the Multipass build/run environment and this report. I verified the work by building and loading the module on the 6.8.0-147 kernel, confirming `qaimd` appears in `tcp_available_congestion_control`, running the full pipeline against real iperf3 servers, and checking that every logged sample reports `cc_used = qaimd` and that server-reported bytes match `tcpi_bytes_acked` to 0.04%. I reviewed all code and results and am prepared to explain the implementation and design choices in the oral exam.